

Кратко в предыдущих сериях

Что советуем перед семинаром освежить в памяти:

- Работа с данными в Питоне: очистка, расчёты, группировки, фильтрации, визуализация сводными таблицами и графиками
- Основы баз данных из курса SQL
- Описательные статистики: среднее, частота абсолютная и относительная, медиана, мода
- Генеральная совокупность, выборка, их отличие
- Парадоксы из предыдущего семинара
- Оценки параметров среднего и пропорции

Тема 2. Работа с данными.

0. Цель и задачи

Цель — научиться работать с данными как аналитики высокого уровня.

Задачи:

- повторить Sanity Check с лекции
- посмотреть на работу с данными на примере
- попрактиковаться в обработке некоторых колонок данных

1. Что сегодня будет

На лекции вы изучали, как работать с данными. На семинаре мы закрепим это на практике

Вспомним пункты Sanity Check с лекции

Вопрос в зал: как правильно работать с данными?

Студенты называют пункты Sanity Check с лекции:

- Ознакомиться с описанием данных — не додумывать, а почитать документацию и разобраться, какая колонка за что отвечает и какие данные там хранятся
- Проверить данные на правдоподобность — конверсии 120% не должно быть. Если вы видите явно некорректные цифры, значит, в данных что-то не так
- Проверить данные на полноту — в том числе и по сегментам
- Проверить, есть ли дубли в данных — если есть, то удалить/обработать их
- При соединении данных убедиться в правильности ключей — тут опять же поможет документация
- Считать метрики так, как это принято у вас в компании — везде должна быть одна и та же методология

2. Кейс: "Анализ поездок Yxxxxx.Такси"

Описание: Компания Yxxxxx запустила сервис такси в Московской области. Есть 4 таблицы: cars (автомобили), drivers (таксисты), clients (клиенты) и orders (заказы). Данные собраны за сентябрь 2024 года.

Задача: изучите данные, постройте статистику по тарифу и сделайте вывод, какие тарифы выгоднее использовать водителям, а какие — компаниям.

```
Ввод [462]: 1 import pandas as pd
2 import numpy as np
3 import scipy.stats as sps
4 import matplotlib.pyplot as plt
5 from tqdm.notebook import tqdm
6 import random
7 from faker import Faker
8 from datetime import date, datetime
9 import ast
10 import re
11
12 import seaborn as sns
13
14 sns.set(font_scale=1, palette='Set2')
15 sns.set_style('whitegrid')
```


Ввод [836]:

```

1 fake = Faker('ru_RU')
2
3 # Города Московской области
4 moscow_region_cities = ["Москва", "Химки", "Мытищи", "Королев", "Подольск", "Люберцы", "Балашиха",
5                          "Железнодорожный", "Реутов", "Долгопрудный"]
6 region_probs = [0.5] + [0.5 / (len(moscow_region_cities) - 1)] * (len(moscow_region_cities) - 1)
7
8 # Современные имена
9 modern_male_names = [
10     "Максим", "Артём", "Михаил", "Даниил", "Александр", "Кирилл", "Матвей",
11     "Иван", "Демид", "Марк", "Егор", "Никита", "Тимофей", "Глеб", "Андрей",
12     "Савелий", "Владислав", "Лев", "Роман", "Степан", "Константин", "Илья",
13     "Ярослав", "Денис", "Федор", "Григорий", "Арсений", "Алексей", "Виктор"
14 ]
15
16 modern_female_names = [
17     "София", "Анна", "Мария", "Виктория", "Алиса", "Полина", "Ева",
18     "Анастасия", "Милана", "Дарья", "Кира", "Арина", "Вероника", "Валерия",
19     "Александра", "Диана", "Елизавета", "Ксения", "Карина", "Ульяна",
20     "Злата", "Василиса", "Яна", "Маргарита", "Алёна", "Стефания", "Наталья"
21 ]
22
23 # Расширенные списки русских имен и фамилий
24 russian_male_names = [
25     "Алексей", "Иван", "Дмитрий", "Олег", "Сергей", "Владимир", "Николай", "Михаил", "Павел",
26 ]
27 russian_female_names = [
28     "Екатерина", "Мария", "Анна", "Ольга", "Татьяна", "Наталья", "Ирина", "Елена", "Светлана"
29 ]
30 russian_surnames_male = [
31     "Иванов", "Петров", "Сидоров", "Кузнецов", "Смирнов", "Попов", "Лебедев", "Козлов", "Новиков",
32 ]
33 russian_surnames_female = [
34     s + "a" if not s.endswith("a") else s
35     for s in russian_surnames_male
36 ] # Иванова, Петрова и т.п.
37
38 # Кавказские имена и фамилии (учитывая разные окончания)
39 caucasian_male_names = [
40     "Аслан", "Магомед", "Рамазан", "Ислам", "Шамиль", "Мурад", "Хасан", "Залим", "Гамид", "Рамазан",
41 ]
42 caucasian_female_names = [
43     "Залина", "Лейла", "Маликат", "Фатима", "Амина", "Зара", "Сафия", "Инса", "Гюльнара", "Айлин",
44 ]
45 # Кавказские фамилии с разными окончаниями
46 caucasian_surnames_male = [
47     "Магомедов", "Гаджиев", "Абдуллаев", "Исаев", "Саидов", "Караев", "Шарипов",
48     "Ахмед", "Багаев", "Джабраилов", "Алиев", "Юсупов", "Абдурахман"
49 ]
50 # Для женских – добавим окончание -а, а также фамилии без изменения (реже)
51 caucasian_surnames_female = [
52     "Магомедова", "Гаджиева", "Абдуллаева", "Исаева", "Саидова", "Караева", "Шарипова",
53     "Ахмед Оглы", "Багаева", "Джабраилова", "Алиева", "Юсупова", "Абдурахман"
54 ]
55
56 # Среднеазиатские имена и фамилии
57 central_asian_male_names = [
58     "Алишер", "Дильшод", "Жомарт", "Нурсултан", "Темирлан", "Рустам", "Бауржан", "Эльнур", "Салтанат",
59 ]
60 central_asian_female_names = [
61     "Махира", "Динара", "Самира", "Гюльнара", "Феруза", "Нигина", "Шахноза", "Диля", "Зухра",
62 ]
63 central_asian_surnames_male = [
64     "Турдиев", "Назаров", "Рахимов", "Алимов", "Саидов", "Жумаев", "Касымов",
65     "Юсупов", "Абдрахманов", "Мирзаев", "Абдуллаев", "Каримов", "Махмудов"
66 ]
67 central_asian_surnames_female = [
68     s + "a" if not s.endswith("a") else s
69     for s in central_asian_surnames_male
70 ]
71
72 ethnic_groups = [
73     {
74         "male_names": russian_male_names,
75         "female_names": russian_female_names,
76         "surnames_male": russian_surnames_male,
77         "surnames_female": russian_surnames_female,
78         "prob": 0.2
79     },
80     {
81         "male_names": caucasian_male_names,

```

```

82     "female_names": caucasian_female_names,
83     "surnames_male": caucasian_surnames_male,
84     "surnames_female": caucasian_surnames_female,
85     "prob": 0.4
86 },
87 {
88     "male_names": central_asian_male_names,
89     "female_names": central_asian_female_names,
90     "surnames_male": central_asian_surnames_male,
91     "surnames_female": central_asian_surnames_female,
92     "prob": 0.4
93 }
94 ]
95
96 def generate_driver_name_and_surname():
97     groups = [g for g in ethnic_groups]
98     probs = [g["prob"] for g in ethnic_groups]
99     group = random.choices(groups, weights=probs, k=1)[0]
100
101     gender = random.choices(["male", "female"], weights=[0.9, 0.1], k=1)[0]
102
103     if gender == "male":
104         first_name = random.choice(group["male_names"])
105         last_name = random.choice(group["surnames_male"])
106     else:
107         first_name = random.choice(group["female_names"])
108         last_name = random.choice(group["surnames_female"])
109
110     return first_name, last_name, gender
111
112
113 def generate_driver():
114     first_name, last_name, gender = generate_driver_name_and_surname()
115     return first_name, last_name, gender
116
117 def random_date_formats(base_date):
118     formats = [
119         "%Y-%m-%d",      # 2024-09-01
120         "%d.%m.%Y",      # 01.09.2024
121         "%d/%m/%Y",      # 01/09/2024
122         "%d-%b-%Y",      # 01-Sep-2024
123         "%Y/%m/%d",      # 2024/09/01
124         "%Y.%m.%d",      # 2024.09.01
125     ]
126     probs = [0.7, 0.05, 0.03, 0.02, 0.1, 0.1]
127     fmt = np.random.choice(formats, p=probs)
128     return base_date.strftime(fmt)
129
130
131 # Параметры
132 n_drivers = 1609
133 n_clients = 97567
134 n_cars = 1813
135
136 # Автомобили
137 tariffs = ['эконом', 'комфорт', 'комфорт+']
138 tariffs_dict = {'эконом': 'э', 'комфорт': 'к', 'комфорт+': 'к+'}
139 tariff_probs_cars = [0.6, 0.2, 0.2]
140 car_models_tariffs = {
141     'эконом': ["Lada Granta", "Renault Logan", "Hyundai Solaris", "Kia Rio", "Volkswagen Polo",
142               "Chery Tiggo 2", "Geely MK", "Datsun on-D0", "Chevrolet Spark", "Fiat Panda",
143               "Suzuki Alto", "Daewoo Matiz"],
144     'комфорт': ["Toyota Corolla", "Hyundai Elantra", "Kia Optima", "Volkswagen Jetta", "Skoda",
145               "Chery Tiggo 4", "Geely Emgrand", "Honda Civic", "Peugeot 408", "Citroen C4",
146               "Opel Astra", "Mazda 3"],
147     'комфорт+': ["Toyota Camry", "Skoda Octavia", "Nissan Qashqai", "Ford Mondeo", "Mazda 6",
148               "Chery Tiggo 7", "Geely Atlas", "Honda Accord", "Volkswagen Passat", "Subaru",
149               "Mitsubishi Outlander", "Peugeot 508"]
150 }
151 probs_tariffs_dict = {
152     'эконом': [0.05, 0.005, 0.005, 0.04, 0.9],
153     'комфорт': [0.03, 0.005, 0.005, 0.02, 0.94],
154     'комфорт+': [0.015, 0.005, 0.005, 0.03, 0.945]
155 }
156 cars = []
157 for i in range(1, n_cars+1):
158     tariff = np.random.choice(tariffs, p=tariff_probs_cars)
159     model = random.choice(car_models_tariffs[tariff])
160     mark = model.split(' ')[0]
161     model = ' '.join(model.split(' ')[1:])
162     if tariff == 'эконом':

```

```

163     rent = random.choice(range(1200, 1801, 50))
164     elif tariff == 'комфорт':
165         rent = random.choice(range(1600, 2501, 50))
166     else:
167         rent = random.choice(range(2000, 3501, 50))
168     cars.append({
169         'id': i,
170         'data': {'марка': mark, 'модель': model, 'тариф': tariff, 'аренда': rent}
171     })
172 cars_df = pd.DataFrame(cars)
173
174 # Таксисты
175 remaining_cars = cars.copy()
176 # print(remaining_cars)
177 drivers = []
178 for i in range(1, n_drivers+1):
179     first_name, last_name, gender = generate_driver()
180     n_workdays = random.choice(np.arange(15, 27))
181     workdays = np.random.choice(pd.date_range('2024-09-01', '2024-09-30', freq='1D'),
182                                 size=n_workdays, replace=False)
183     car = random.choice(remaining_cars)
184     # print(car)
185     remaining_cars.remove(car)
186     car_tariff = cars[cars['id'] - 1]['data']['тариф']
187     drivers.append({
188         'id': i,
189         'first_name': first_name,
190         'last_name': last_name,
191         'auto': car['id'],
192         'tariff': car_tariff,
193         'rating': round(random.uniform(3.0, 5.0), 2),
194         'city': np.random.choice(moscow_region_cities, p=region_probs),
195         'workdays': workdays
196     })
197 drivers_df = pd.DataFrame(drivers)
198
199 # Клиенты
200 clients = []
201 for i in range(1, n_clients+1):
202     gender = np.random.choice(['male', 'female'])
203     first_name = random.choice(modern_male_names) if gender == 'male' else random.choice(modern_female_names)
204     last_name = fake.last_name_male() if gender == 'male' else fake.last_name_female()
205     clients.append({
206         'id': i,
207         'first_name': first_name,
208         'last_name': last_name,
209         'city': np.random.choice(moscow_region_cities),
210         'rating': round(random.uniform(3.0, 5.0), 2)
211     })
212 clients_df = pd.DataFrame(clients)
213
214 # Заказы
215 clients_df['n_orders'] = np.random.poisson(3.5, size=n_clients)
216
217 orders = []
218 order_id = 1
219 for idx, client in tqdm(clients_df.iterrows()):
220     for _ in range(client['n_orders']):
221         tariff_probs = [0.7, 0.16, 0.14]
222         tariff = np.random.choice(tariffs, p=tariff_probs)
223         # print(tariff)
224         t_drivers = drivers_df if random.random() < 0.05 else drivers_df.loc[drivers_df['tariff'] == tariff]
225         driver = t_drivers.loc[random.choice(t_drivers.index)]
226         car = random.choice(cars_df['id']) if random.random() < 0.02 else driver['auto']
227         # print(car)
228         p_r = probs_tariffs_dict[tariff]
229         # print(cars_df.loc[cars_df['id'] == car]['data'].iloc[0])
230         auto_tariff = cars[cars['id'] - 1]['data']['тариф']
231         tariff_levels = {'эконом': 1, 'комфорт': 2, 'комфорт+': 3}
232         if tariff_levels[auto_tariff] > tariff_levels[tariff]:
233             tariff = auto_tariff
234         # print(tariff)
235         coef = round(random.uniform(0.8, 1.4), 2)
236         base_cost = {'эконом': 150, 'комфорт': 200, 'комфорт+': 250}[tariff]
237         len_cost = {'эконом': 10, 'комфорт': 18, 'комфорт+': 21}[tariff]
238         time_cost = {'эконом': 9, 'комфорт': 15, 'комфорт+': 18}[tariff]
239         len_distr = {'эконом': sps.gamma(a=2.5, scale=10),
240                     'комфорт': sps.gamma(a=3, scale=9),
241                     'комфорт+': sps.gamma(a=3.5, scale=8)}
242
243         length = len_distr[tariff].rvs()

```

```

244     if random.random() < 0.05:
245         tariff = np.nan
246     elif random.random() < 0.1:
247         tariff = tariffs_dict[tariff]
248
249     time = length * random.uniform(1.0, 2.0) # Время связано с расстоянием с разбросом
250     cost = max(base_cost, round(coef * (len_cost * length + time_cost * time)))
251     if random.random() < 0.1:
252         length *= 1000
253         length = f"{round(length, np.random.choice(np.arange(2)))) m"
254     else:
255         length = f"{round(length, np.random.choice(np.arange(3)))) km"
256
257     if random.random() < 0.15:
258         time *= 60 # иногда в секундах
259         time = f"{round(time, np.random.choice(np.arange(2)))) sec"
260     else:
261         time = f"{round(time, np.random.choice(np.arange(3)))) min"
262
263     rnd_taxist = random.random()
264     rnd_cl = random.random()
265     taxist = np.nan if rnd_taxist > 0.99 else driver['id'] if rnd_taxist < 0.95 else f"{c
266     client_id = np.nan if rnd_cl > 0.99 else client['id'] if rnd_cl < 0.95 else f"{client
267
268     # Оценки с вероятностями
269     percent_map = {1:20, 2:40, 3:60, 4:80, 5:100}
270     values = [1, 2, 3, 4, 5]
271
272     if random.random() < 0.02:
273         r1 = percent_map[np.random.choice(values, p=p_r)]
274     else:
275         r1 = np.random.choice(values, p=p_r)
276
277     if random.random() < 0.02:
278         r2 = percent_map[np.random.choice(values, p=p_probs)]
279     else:
280         r2 = np.random.choice(values, p=p_probs)
281
282     used_car = driver['auto'] if random.random() > 0.05 else random.choice(cars)['id']
283
284 #     print(datetime(np.random.choice(driver['workdays'])))
285     orders.append({
286         'id': order_id,
287         'data': random_date_formats(pd.to_datetime(np.random.choice(driver['workdays'])))
288         'taxist': taxist,
289         'client': client_id,
290         'r1': r1,
291         'r2': r2,
292         'tariff': tariff,
293         'len': length,
294         'time': time,
295         'coef': coef,
296         'cost': cost,
297         'auto': used_car,
298         'service_fee': cost * 0.2
299     })
300     order_id += 1
301 #     break
302
303 orders_df = pd.DataFrame(orders)
304
305 # Сопоставим только ID
306 orders_df['taxist_id'] = pd.to_numeric(orders_df['taxist'], errors='coerce')
307 orders_df['client_id'] = pd.to_numeric(orders_df['client'], errors='coerce')
308
309 # Таксисты
310 driver_ratings = orders_df.groupby('taxist_id')['r1'].mean()
311 drivers_df['rating'] = drivers_df['id'].map(driver_ratings)
312 orders_df.drop(columns=['taxist_id'], inplace=True)
313 drivers_df.drop(columns=['workdays'], inplace=True)
314 drivers_df.drop(columns=['tariff'], inplace=True)
315
316 # Клиенты
317 client_ratings = orders_df.groupby('client_id')['r2'].mean()
318 clients_df['rating'] = clients_df['id'].map(client_ratings)
319 orders_df.drop(columns=['client_id'], inplace=True)
320
321 # Добавляем пропуски в оценки
322 nan_r1_idx = orders_df.sample(frac=0.85).index
323 nan_r2_idx = orders_df.sample(frac=0.5).index
324 orders_df.loc[nan_r1_idx, 'r1'] = np.nan

```

```
325 orders_df.loc[nan_r2_idx, 'r2'] = np.nan
326
327 # Сохраняем
328 cars_df.to_csv('cars.csv', index=False)
329 drivers_df.to_csv('drivers.csv', index=False)
330 clients_df.to_csv('clients.csv', index=False)
331 orders_df.to_csv('orders.csv', index=False)
```

97567/? [07:16<00:00, 241.55it/s]

2.1 Sanity check 0. Ознакомиться с описанием данных

Посмотрим на описание и сэмплы данных — оно в отдельном файле

Вопрос в зал: что не так с датасетом?

Отметим ряд проблем

- Оценки смешаны: часть в процентах, часть по 5-балльной шкале
- Расстояние и время поездки в разных единицах
- Дата в разном формате
- Поля `taxist` и `client` — иногда ID, иногда строка с именем
- Автомобиль в заказе может не совпадать с текущим авто таксиста. Это происходит из-за того, что таксист мог арендовать разные автомобили
- Данные по авто указаны в json-формате
- Пропуски в оценках и рейтингах

Вопрос в зал: что будем делать?

План предобработки:

- Преобразовать оценки: проценты перевести в баллы (80% → 4)
- Привести длину маршрута к километрам, время к минутам (удалить строки типа " km", делить метры на 1000, секунды на 60)
- Преобразовать дату к одному формату
- Поля `taxist` и `client`: всё привести к ID. Если имя — попробовать сопоставить с таблицами. Если не получается — исключить такие строчки
- Преобразовать данные по авто с json в таблицу
- Проверить авто поездок: оставить актуальные или пометить старые
- Пропуски: оставить NaN
- Пересчитать рейтинги водителей и клиентов по среднему от заказов

Загрузим данные

Ввод [865]:

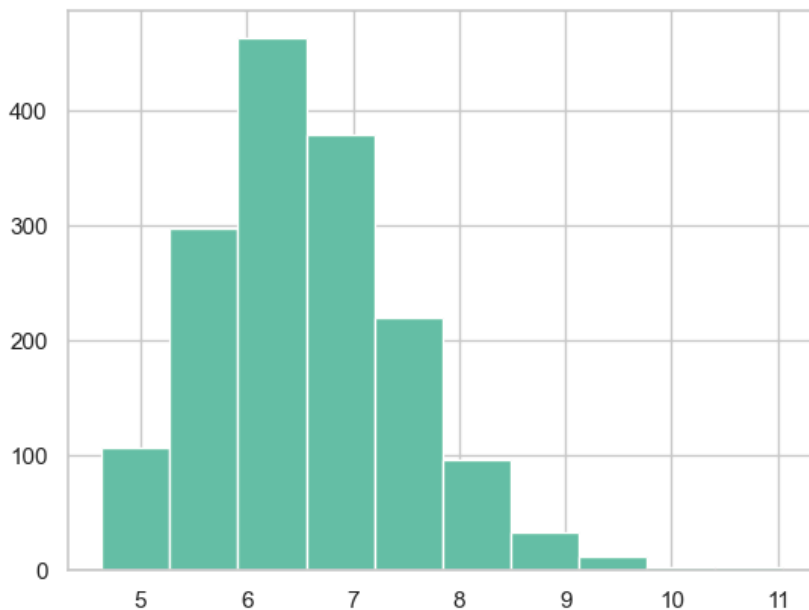
```
1 cars_df = pd.read_csv('cars.csv')
2 drivers_df = pd.read_csv('drivers.csv')
3 clients_df = pd.read_csv('clients.csv')
4 orders_df = pd.read_csv('orders.csv')
```

2.2 Sanity check 1. Правдоподобность

Оценим правдоподобность данных по средней оценке водителей

```
Ввод [866]: 1 drivers_df['rating'].hist()
```

```
Out[866]: <Axes: >
```



Странно, рейтинг распределен около 7, что же не так с данными

Вопрос в зал: почему так?

Ответ: из-за разного формата оценок (1-5 или процентный)

2.2 Sanity check 1. Полнота данных

Посмотрим на полноту данных заказов, а именно, на айди клиентов.

```
Ввод [867]: 1 orders_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 341309 entries, 0 to 341308
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               341309 non-null  int64
1   data            341309 non-null  object
2   taxist          337855 non-null  object
3   client          337916 non-null  object
4   r1              51196 non-null   float64
5   r2              170655 non-null  float64
6   tariff          324173 non-null  object
7   len             341309 non-null  object
8   time            341309 non-null  object
9   coef            341309 non-null  float64
10  cost            341309 non-null  int64
11  auto            341309 non-null  int64
12  service_fee     341309 non-null  float64
dtypes: float64(4), int64(3), object(6)
memory usage: 33.9+ MB
```

Полнота 99%, но тип object. Посмотрим, что внутри данных

```
Ввод [869]: 1 orders_df['client'].head(30)
```

```
Out[869]: 0      1
1      1
2      1
3      2
4      2
5      2
6      2
7      2
8      3
9      3
10     4
11     4
12     4
13     5
14     5
15     5
16     5
17     5
18     6
19     6
20     7
21     7
22     7
23     8
24     8
25     8
26     8
27     Арина С.
28     8
29     NaN
Name: client, dtype: object
```

Теперь понятно: у нас вместо айди клиента иногда указано его имя.

Вопрос в зал: стоит ли заниматься "угадыванием" клиента по имени?

Ответ давайте оценим количество таких данных:

```
Ввод [872]: 1 (orders_df['client'].apply(lambda x: str(x).isdigit())).mean()
```

```
Out[872]: 0.9502239905774533
```

95% данных корректны, при этом при "угадывании" мы можем получить некорректные данные. Вывод — угадывать не нужно, стоит заменить некорректные данные на пропуски

2.4 Sanity check 3. Дубли

Посмотрим на дубли по айди заказа

```
Ввод [873]: 1 len(orders_df), len(orders_df['id'].unique())
```

```
Out[873]: (341309, 341309)
```

Дублей нет, и на том спасибо 😊

Обработка данных

```

Ввод [838]: 1 # Преобразуем оценки
2 def clean_rating(x):
3     if pd.isna(x):
4         return np.nan
5     x = float(x)
6     if x > 5:
7         return x / 20
8     return x
9
10 orders_df['r1'] = orders_df['r1'].apply(clean_rating)
11 orders_df['r2'] = orders_df['r2'].apply(clean_rating)
12
13 orders_df

```

Out [838]:

	id	data	taxist	client	r1	r2	tariff	len	time	coef	cost	auto	service_fee
0	1	2024-09-16	151	1	NaN	5.0	эконом	10.0 km	19.0 min	1.09	293	443	58.6
1	2	2024/09/08	511	1	NaN	5.0	эконом	11.7 km	22.0 min	0.92	286	1479	57.2
2	3	2024-09-14	632	1	NaN	5.0	эконом	14.16 km	18.14 min	1.27	387	717	77.4
3	4	2024-09-03	1439	2	NaN	5.0	комфорт	27.6 km	45.0 min	1.37	1597	1241	319.4
4	5	2024-09-19	292	2	5.0	NaN	комфорт	44.6 km	72.33 min	0.92	1737	1646	347.4
...	...	...	...	...	...	...	...	...	...	...	...	...	...
341304	341305	21.09.2024	1288	97566	NaN	5.0	комфорт	37.86 km	65.3 min	1.21	2010	245	402.0
341305	341306	2024-09-17	1421	97567	5.0	5.0	эконом	59.8 km	84.5 min	1.10	1494	1094	298.8
341306	341307	2024-09-09	680	97567	NaN	5.0	эконом	20.24 km	40.2 min	1.28	722	1759	144.4
341307	341308	2024-09-01	1039	97567	NaN	5.0	эконом	6.0 km	9.0 min	1.30	179	325	35.8
341308	341309	2024-09-05	584	97567	5.0	NaN	эконом	26117.8 m	34.4 min	0.85	485	1551	97.0

341309 rows × 13 columns

```

Ввод [839]: 1 # Преобразуем длину маршрута
2 def clean_length(x):
3     if pd.isna(x):
4         return np.nan
5     if isinstance(x, str):
6         if 'km' in x:
7             return float(re.sub(r'^0-9.', '', x))
8         elif 'm' in x:
9             return float(re.sub(r'^0-9.', '', x)) / 1000
10    else:
11        return float(x)
12    return float(x)
13
14 orders_df['len'] = orders_df['len'].apply(clean_length)
15 orders_df

```

Out [839]:

	id	data	taxist	client	r1	r2	tariff	len	time	coef	cost	auto	service_fee
0	1	2024-09-16	151	1	NaN	5.0	эконом	10.0000	19.0 min	1.09	293	443	58.6
1	2	2024/09/08	511	1	NaN	5.0	эконом	11.7000	22.0 min	0.92	286	1479	57.2
2	3	2024-09-14	632	1	NaN	5.0	эконом	14.1600	18.14 min	1.27	387	717	77.4
3	4	2024-09-03	1439	2	NaN	5.0	комфорт	27.6000	45.0 min	1.37	1597	1241	319.4
4	5	2024-09-19	292	2	5.0	NaN	комфорт	44.6000	72.33 min	0.92	1737	1646	347.4
...	...	...	...	...	...	...	...	...	...	...	...	...	...
341304	341305	21.09.2024	1288	97566	NaN	5.0	комфорт	37.8600	65.3 min	1.21	2010	245	402.0
341305	341306	2024-09-17	1421	97567	5.0	5.0	эконом	59.8000	84.5 min	1.10	1494	1094	298.8
341306	341307	2024-09-09	680	97567	NaN	5.0	эконом	20.2400	40.2 min	1.28	722	1759	144.4
341307	341308	2024-09-01	1039	97567	NaN	5.0	эконом	6.0000	9.0 min	1.30	179	325	35.8
341308	341309	2024-09-05	584	97567	5.0	NaN	эконом	26.1178	34.4 min	0.85	485	1551	97.0

341309 rows × 13 columns

👉 Практика: преобразование времени поездки

Ввод [840]:

```

1 # Преобразуем время поездки
2 def clean_time(x):
3     # напишите код <...>
4     if pd.isna(x):
5         return np.nan
6     if isinstance(x, str):
7         if 'min' in x:
8             return float(re.sub(r'^0-9.', '', x))
9         elif 'sec' in x:
10            return float(re.sub(r'^0-9.', '', x)) / 60
11        else:
12            return float(x)
13    return float(x)
14
15 orders_df['time'] = orders_df['time'].apply(clean_time)
16 orders_df

```

Out[840]:

	id	data	taxist	client	r1	r2	tariff	len	time	coef	cost	auto	service_fee
0	1	2024-09-16	151	1	NaN	5.0	эконом	10.0000	19.00	1.09	293	443	58.6
1	2	2024/09/08	511	1	NaN	5.0	эконом	11.7000	22.00	0.92	286	1479	57.2
2	3	2024-09-14	632	1	NaN	5.0	эконом	14.1600	18.14	1.27	387	717	77.4
3	4	2024-09-03	1439	2	NaN	5.0	комфорт	27.6000	45.00	1.37	1597	1241	319.4
4	5	2024-09-19	292	2	5.0	NaN	комфорт	44.6000	72.33	0.92	1737	1646	347.4
...	...	...	...	...	...	...	...	...	...	...	...	...	...
341304	341305	21.09.2024	1288	97566	NaN	5.0	комфорт	37.8600	65.30	1.21	2010	245	402.0
341305	341306	2024-09-17	1421	97567	5.0	5.0	эконом	59.8000	84.50	1.10	1494	1094	298.8
341306	341307	2024-09-09	680	97567	NaN	5.0	эконом	20.2400	40.20	1.28	722	1759	144.4
341307	341308	2024-09-01	1039	97567	NaN	5.0	эконом	6.0000	9.00	1.30	179	325	35.8
341308	341309	2024-09-05	584	97567	5.0	NaN	эконом	26.1178	34.40	0.85	485	1551	97.0

341309 rows × 13 columns

```

Ввод [841]: 1 # Преобразуем время
2 # Список форматов, которые мы ожидаем
3 date_formats = [
4     "%Y-%m-%d",      # 2024-09-01
5     "%d.%m.%Y",      # 01.09.2024
6     "%d/%m/%Y",      # 01/09/2024
7     "%d-%b-%Y",      # 01-Sep-2024
8     "%Y/%m/%d",      # 2024/09/01
9     "%Y.%m.%d",      # 2024.09.01
10 ]
11
12 def parse_date_with_formats(date_str):
13     if pd.isna(date_str):
14         return pd.NaT
15     for fmt in date_formats:
16         try:
17             return datetime.strptime(date_str, fmt)
18         except (ValueError, TypeError):
19             continue
20     # Если ни один формат не подошёл, вернём NaT
21     return pd.NaT
22
23
24 orders_df['data'] = pd.to_datetime(orders_df['data'].apply(parse_date_with_formats))
25 orders_df

```

Out[841]:

	id	data	taxist	client	r1	r2	tariff	len	time	coef	cost	auto	service_fee
0	1	2024-09-16	151	1	NaN	5.0	эконом	10.0000	19.00	1.09	293	443	58.6
1	2	2024-09-08	511	1	NaN	5.0	эконом	11.7000	22.00	0.92	286	1479	57.2
2	3	2024-09-14	632	1	NaN	5.0	эконом	14.1600	18.14	1.27	387	717	77.4
3	4	2024-09-03	1439	2	NaN	5.0	комфорт	27.6000	45.00	1.37	1597	1241	319.4
4	5	2024-09-19	292	2	5.0	NaN	комфорт	44.6000	72.33	0.92	1737	1646	347.4
...	...	...	...	...	...	...	...	...	...	...	...	...	...
341304	341305	2024-09-21	1288	97566	NaN	5.0	комфорт	37.8600	65.30	1.21	2010	245	402.0
341305	341306	2024-09-17	1421	97567	5.0	5.0	эконом	59.8000	84.50	1.10	1494	1094	298.8
341306	341307	2024-09-09	680	97567	NaN	5.0	эконом	20.2400	40.20	1.28	722	1759	144.4
341307	341308	2024-09-01	1039	97567	NaN	5.0	эконом	6.0000	9.00	1.30	179	325	35.8
341308	341309	2024-09-05	584	97567	5.0	NaN	эконом	26.1178	34.40	0.85	485	1551	97.0

341309 rows × 13 columns

```

Ввод [842]: 1 # Очистим ID таксиста и клиента
2 orders_df['taxist'] = pd.to_numeric(orders_df['taxist'].apply(lambda x: x if str(x).isdigit()
3 orders_df['client'] = pd.to_numeric(orders_df['client'].apply(lambda x: x if str(x).isdigit()

```

Ввод [843]:

```

1 # Пересчитаем рейтинги
2 driver_ratings = orders_df.groupby('taxist')[['r1']].mean().reset_index().astype(float)
3 client_ratings = orders_df.groupby('client')[['r2']].mean().reset_index().astype(float)
4
5 drivers_df = drivers_df.merge(driver_ratings, how='left', left_on='id', right_on='taxist')
6 drivers_df['rating'] = drivers_df['r1']
7 drivers_df.drop(columns=['taxist', 'r1'], inplace=True)
8
9 clients_df = clients_df.merge(client_ratings, how='left', left_on='id', right_on='client')
10 clients_df['rating'] = clients_df['r2']
11 clients_df.drop(columns=['client', 'r2'], inplace=True)
12
13 drivers_df

```

Out[843]:

	id	first_name	last_name	auto	rating	city
0	1	Нурсултан	Жумаев	1494	4.964286	Реутов
1	2	Шамиль	Абдурахман	1798	5.000000	Москва
2	3	Мурад	Абдурахман	850	4.714286	Железнодорожный
3	4	Рустам	Назаров	940	4.800000	Долгопрудный
4	5	Дильшод	Касымов	1439	4.687500	Москва
...	...	...	...	...	...	...
1604	1605	Нурсултан	Алимов	982	4.727273	Москва
1605	1606	Алишер	Юсупов	1730	4.809524	Москва
1606	1607	Алишер	Жумаев	183	4.731707	Москва
1607	1608	Магомед	Исаев	1755	4.763158	Железнодорожный
1608	1609	Владимир	Новиков	565	4.739130	Москва

1609 rows × 6 columns

👉 Практика: преобразование тарифов

Ввод [844]:

```

1 # Преобразуем тарифы
2 # напишите код <...>
3 tariff_dict = {'э': 'эконом', 'к': 'комфорт', 'к+': 'комфорт+'}
4
5 orders_df['tariff'] = orders_df['tariff'].replace(tariff_dict)
6 orders_df

```

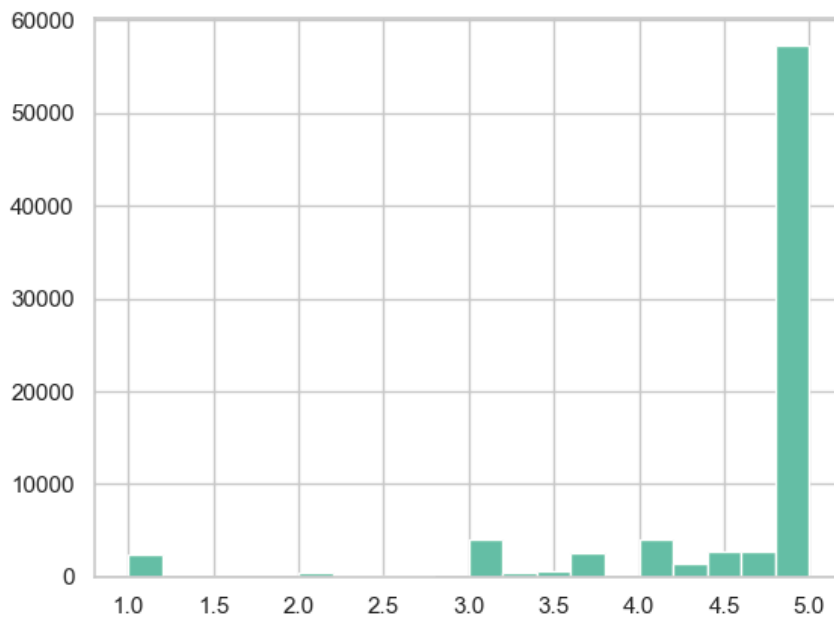
Out[844]:

	id	data	taxist	client	r1	r2	tariff	len	time	coef	cost	auto	service_fee
0	1	2024-09-16	151.0	1.0	NaN	5.0	эконом	10.0000	19.00	1.09	293	443	58.6
1	2	2024-09-08	511.0	1.0	NaN	5.0	эконом	11.7000	22.00	0.92	286	1479	57.2
2	3	2024-09-14	632.0	1.0	NaN	5.0	эконом	14.1600	18.14	1.27	387	717	77.4
3	4	2024-09-03	1439.0	2.0	NaN	5.0	комфорт	27.6000	45.00	1.37	1597	1241	319.4
4	5	2024-09-19	292.0	2.0	5.0	NaN	комфорт	44.6000	72.33	0.92	1737	1646	347.4
...	...	...	...	...	...	...	...	...	...	...	...	...	...
341304	341305	2024-09-21	1288.0	97566.0	NaN	5.0	комфорт	37.8600	65.30	1.21	2010	245	402.0
341305	341306	2024-09-17	1421.0	97567.0	5.0	5.0	эконом	59.8000	84.50	1.10	1494	1094	298.8
341306	341307	2024-09-09	680.0	97567.0	NaN	5.0	эконом	20.2400	40.20	1.28	722	1759	144.4
341307	341308	2024-09-01	1039.0	97567.0	NaN	5.0	эконом	6.0000	9.00	1.30	179	325	35.8
341308	341309	2024-09-05	584.0	97567.0	5.0	NaN	эконом	26.1178	34.40	0.85	485	1551	97.0

341309 rows × 13 columns

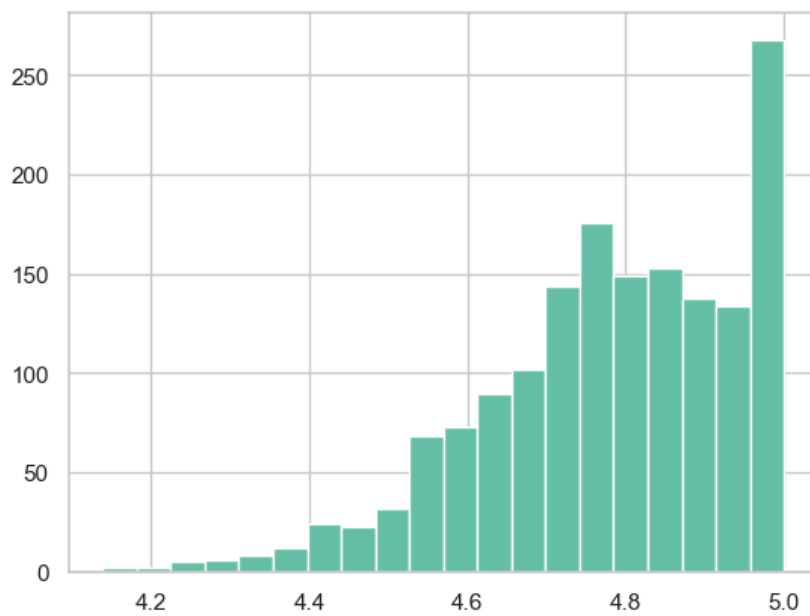
```
Ввод [845]: 1 clients_df['rating'].hist(bins=20)
```

Out[845]: <Axes: >



```
Ввод [846]: 1 drivers_df['rating'].hist(bins=20)
```

Out[846]: <Axes: >



```
Ввод [847]: 1 # обрабатываем данные по авто
2 def process_cars(x):
3     return x['data']['марка'], x['data']['модель'], x['data']['тариф'], x['data']['аренда']
4
5 cars_df['data'] = cars_df['data'].apply(ast.literal_eval)
6 cars_df[['mark', 'model', 'tariff', 'rent_cost']] = cars_df.apply(process_cars, axis=1, result_type='expand')
7 cars_df.drop(columns=['data'], inplace=True)
8 cars_df
```

Out[847]:

	id	mark	model	tariff	rent_cost
0	1	Lada	Granta	эконом	1550
1	2	Daewoo	Matiz	эконом	1450
2	3	Chevrolet	Spark	эконом	1550
3	4	Mitsubishi	Outlander	комфорт+	3250
4	5	Volkswagen	Passat	комфорт+	2600
...	...	...	...	...	...
1808	1809	Renault	Logan	эконом	1700
1809	1810	Lada	Granta	эконом	1700
1810	1811	Peugeot	508	комфорт+	3400
1811	1812	Chery	Tiggo 2	эконом	1750
1812	1813	Kia	Rio	эконом	1650

1813 rows × 5 columns

Ввод [848]: 1 cars_df

Out[848]:

	id	mark	model	tariff	rent_cost
0	1	Lada	Granta	эконом	1550
1	2	Daewoo	Matiz	эконом	1450
2	3	Chevrolet	Spark	эконом	1550
3	4	Mitsubishi	Outlander	комфорт+	3250
4	5	Volkswagen	Passat	комфорт+	2600
...	...	...	...	...	...
1808	1809	Renault	Logan	эконом	1700
1809	1810	Lada	Granta	эконом	1700
1810	1811	Peugeot	508	комфорт+	3400
1811	1812	Chery	Tiggo 2	эконом	1750
1812	1813	Kia	Rio	эконом	1650

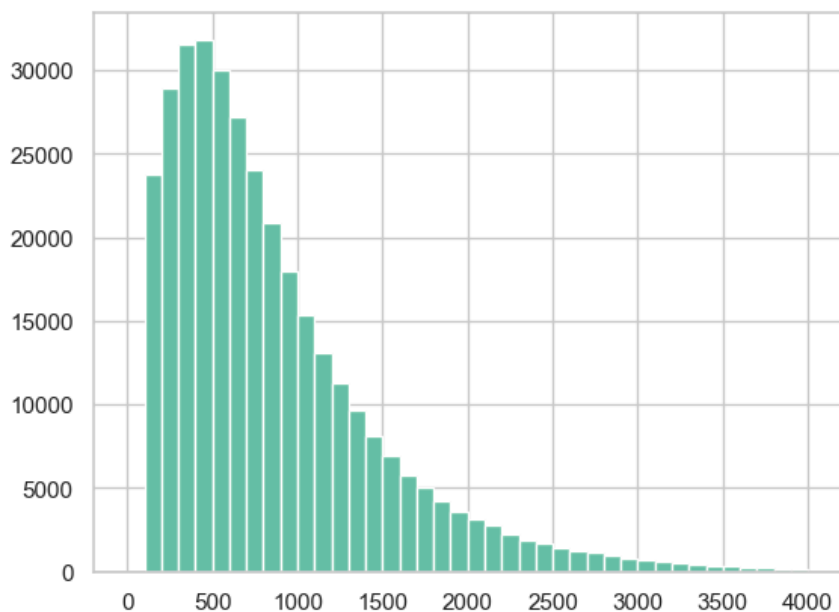
1813 rows × 5 columns

Ввод [849]: 1 orders_df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 341309 entries, 0 to 341308
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               341309 non-null  int64
1   data             341309 non-null  datetime64[ns]
2   taxist           324253 non-null  float64
3   client           324320 non-null  float64
4   r1               51196 non-null   float64
5   r2               170655 non-null  float64
6   tariff           324173 non-null  object
7   len              341309 non-null  float64
8   time             341309 non-null  float64
9   coef             341309 non-null  float64
10  cost              341309 non-null  int64
11  auto              341309 non-null  int64
12  service_fee      341309 non-null  float64
dtypes: datetime64[ns](1), float64(8), int64(3), object(1)
memory usage: 33.9+ MB
```

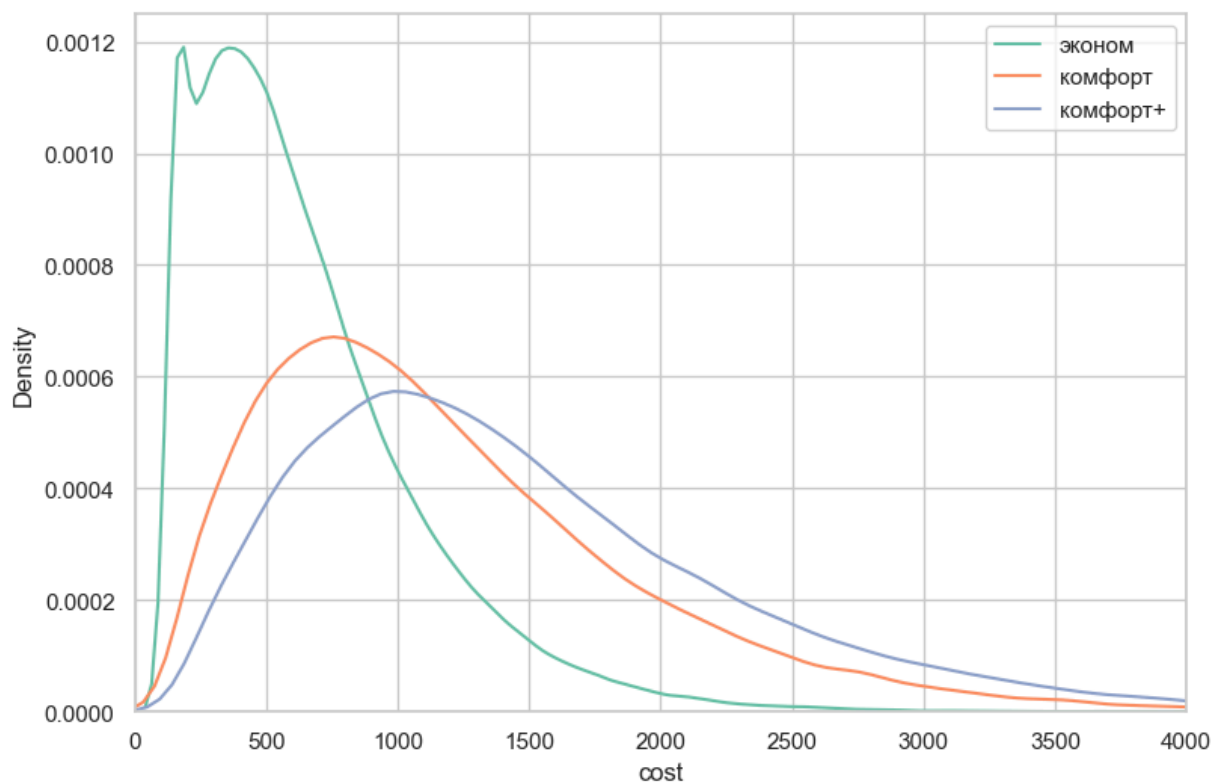
```
Ввод [850]: 1 orders_df['cost'].hist(bins=np.linspace(0, 4000, 41))
```

```
Out [850]: <Axes: >
```



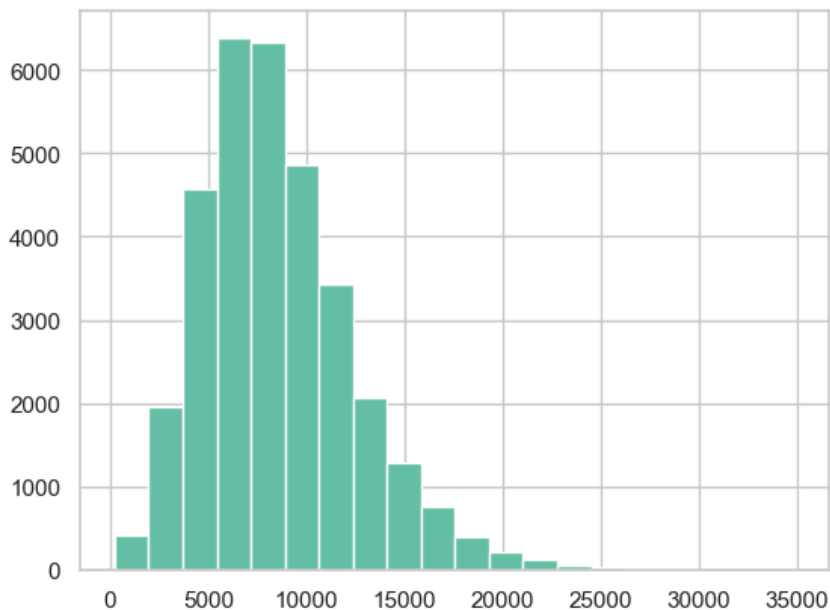
Вернемся к заданию: постройте статистику по тарифам и сделайте вывод, какие тарифы выгоднее использовать водителям, а какие — компании.

```
Ввод [851]: 1 plt.figure(figsize=(9, 6))
2 orders_econom = orders_df[orders_df['tariff'] == 'эконом']
3 orders_comfort = orders_df[orders_df['tariff'] == 'комфорт']
4 orders_comfort_plus = orders_df[orders_df['tariff'] == 'комфорт+']
5
6 sns.kdeplot(orders_econom['cost'], label='эконом')
7 sns.kdeplot(orders_comfort['cost'], label='комфорт')
8 sns.kdeplot(orders_comfort_plus['cost'], label='комфорт+')
9 plt.xlim((0, 4000))
10 plt.legend()
11 plt.show()
```



```
Ввод [852]: 1 orders_group = orders_df.groupby(['taxist', 'data']).agg({
2           'id': 'count',
3           'cost': 'sum',
4           'service_fee': 'sum'
5         }).reset_index()
6 orders_group['cost'].hist(bins=20)
```

Out [852]: <Axes: >



2.5 Sanity check 4. Неправильные ключи

Вопрос в зал: мы хотим сегментировать заказы по тарифам автомобилей и посчитать доходы таксиста и доходы компании от работы таксистов. Как нам это сделать?

Ответ:

- доход водителя — это выручка - комиссия - аренда автомобиля
- доход компании — это комиссия + аренда автомобиля
- к сгруппированным данным надо приджойнить водителя, а потом — к его авто присоединить данные по авто
- как соединять — изучаем таблички

```
Ввод [853]: 1 orders_group = orders_group.merge(drivers_df[['id', 'auto']], left_on='taxist', right_on='id'
2 orders_group = orders_group.merge(cars_df[['id', 'tariff', 'rent_cost']], left_on='auto', rig
3 orders_group['driver_income'] = orders_group['cost'] - orders_group['service_fee'] - orders_g
4 orders_group['company_income'] = orders_group['service_fee'] + orders_group['rent_cost']
5 orders_group
```

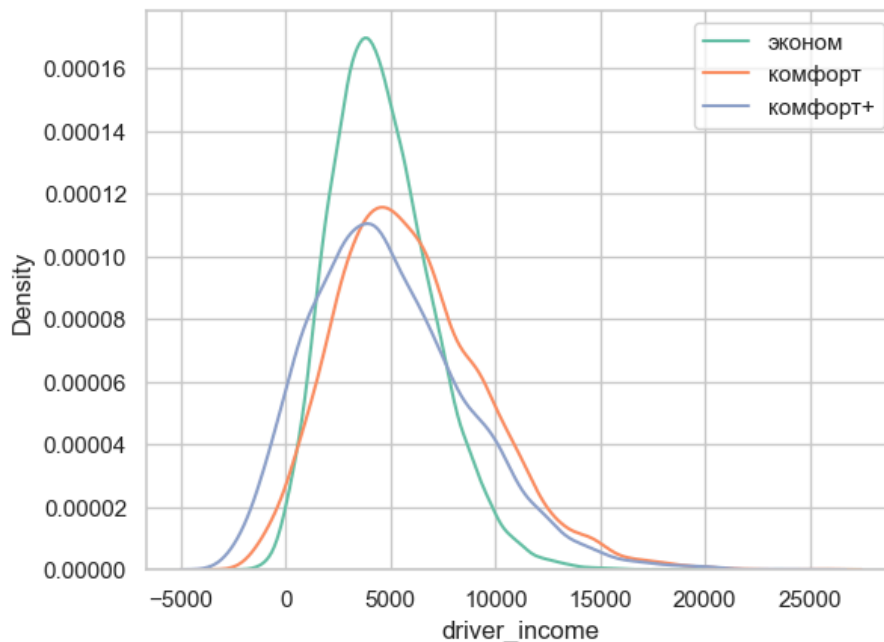
Out [853]:

	taxist	data	id_x	cost	service_fee	id_y	auto	id	tariff	rent_cost	driver_income	company_income
0	1.0	2024-09-01	12	8674	1734.8	1	1494	1494	эконом	1200	5739.2	2934.8
1	1.0	2024-09-02	10	8136	1627.2	1	1494	1494	эконом	1200	5308.8	2827.2
2	1.0	2024-09-03	9	6361	1272.2	1	1494	1494	эконом	1200	3888.8	2472.2
3	1.0	2024-09-04	13	9576	1915.2	1	1494	1494	эконом	1200	6460.8	3115.2
4	1.0	2024-09-05	7	3664	732.8	1	1494	1494	эконом	1200	1731.2	1932.8
...	...	...	...	...	...	...	...	...	...	...	...	...
32909	1609.0	2024-09-19	14	21130	4226.0	1609	565	565	комфорт	2400	14504.0	6626.0
32910	1609.0	2024-09-22	8	10786	2157.2	1609	565	565	комфорт	2400	6228.8	4557.2
32911	1609.0	2024-09-24	13	15341	3068.2	1609	565	565	комфорт	2400	9872.8	5468.2
32912	1609.0	2024-09-25	8	6322	1264.4	1609	565	565	комфорт	2400	2657.6	3664.4
32913	1609.0	2024-09-30	14	15372	3074.4	1609	565	565	комфорт	2400	9897.6	5474.4

32914 rows × 12 columns

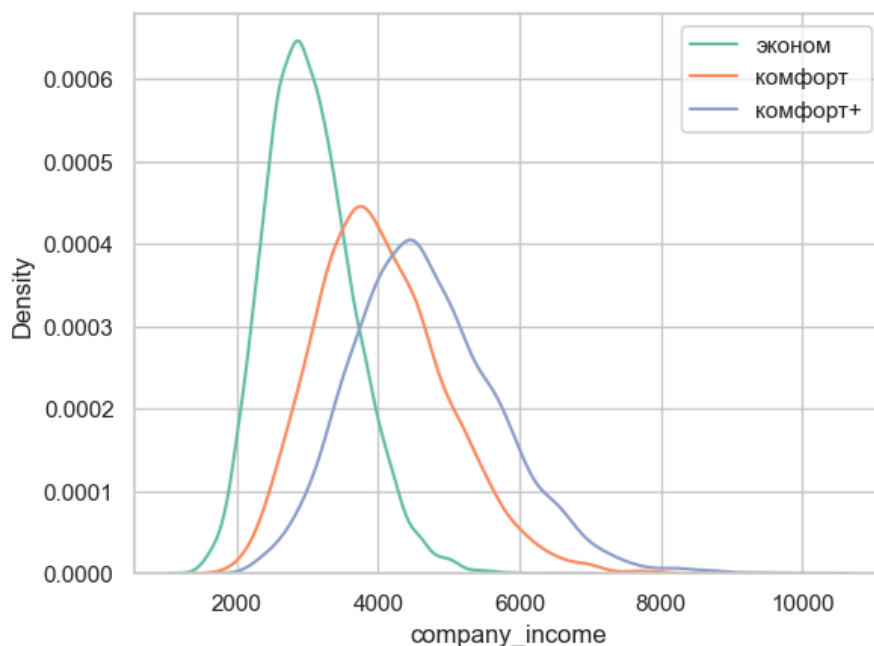
Строим оценку плотности распределения доходов водителей

```
Ввод [854]: 1 orders_group_econom = orders_group[orders_group['tariff'] == 'эконом']
2 orders_group_comfort = orders_group[orders_group['tariff'] == 'комфорт']
3 orders_group_comfort_plus = orders_group[orders_group['tariff'] == 'комфорт+']
4
5 sns.kdeplot(orders_group_econom['driver_income'], label='эконом')
6 sns.kdeplot(orders_group_comfort['driver_income'], label='комфорт')
7 sns.kdeplot(orders_group_comfort_plus['driver_income'], label='комфорт+')
8 # plt.xlim((0, 4000))
9 plt.legend()
10 plt.show()
```



Строим оценку плотности распределения доходов компании с 1 водителем

```
Ввод [856]: 1 orders_group_econom = orders_group[orders_group['tariff'] == 'эконом']
2 orders_group_comfort = orders_group[orders_group['tariff'] == 'комфорт']
3 orders_group_comfort_plus = orders_group[orders_group['tariff'] == 'комфорт+']
4
5 sns.kdeplot(orders_group_econom['company_income'], label='эконом')
6 sns.kdeplot(orders_group_comfort['company_income'], label='комфорт')
7 sns.kdeplot(orders_group_comfort_plus['company_income'], label='комфорт+')
8 # plt.xlim((0, 4000))
9 plt.legend()
10 plt.show()
```



Оценим также, сколько компания зарабатывает со всех водителей с определенным тарифом в день

Ввод [857]:

```
1 orders_group_tariff = orders_group.groupby(['data', 'tariff'])['company_income'].sum().reset_
2 orders_group_tariff
```

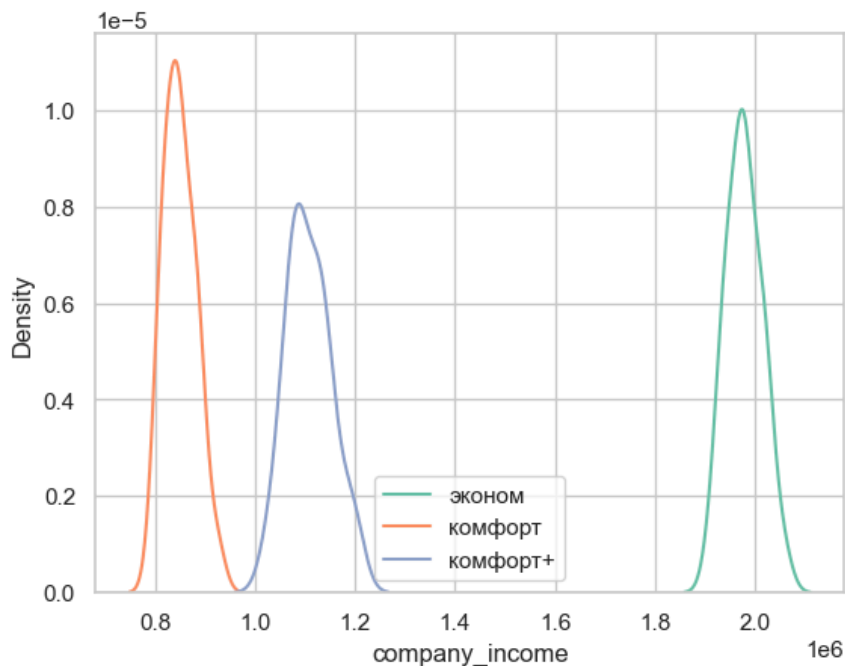
Out[857]:

	data	tariff	company_income
0	2024-09-01	комфорт	859080.0
1	2024-09-01	комфорт+	1075317.6
2	2024-09-01	эконом	1966637.6
3	2024-09-02	комфорт	845588.2
4	2024-09-02	комфорт+	1122041.8
...	...	...	...
85	2024-09-29	комфорт+	1074340.2
86	2024-09-29	эконом	2015050.8
87	2024-09-30	комфорт	837737.4
88	2024-09-30	комфорт+	1119389.4
89	2024-09-30	эконом	2027076.8

90 rows × 3 columns

Ввод [858]:

```
1 orders_group_tariff_econom = orders_group_tariff[orders_group_tariff['tariff'] == 'эконом']
2 orders_group_tariff_comfort = orders_group_tariff[orders_group_tariff['tariff'] == 'комфорт']
3 orders_group_tariff_comfort_plus = orders_group_tariff[orders_group_tariff['tariff'] == 'комфорт+']
4
5 sns.kdeplot(orders_group_tariff_econom['company_income'], label='эконом')
6 sns.kdeplot(orders_group_tariff_comfort['company_income'], label='комфорт')
7 sns.kdeplot(orders_group_tariff_comfort_plus['company_income'], label='комфорт+')
8 # plt.xlim((0, 4000))
9 plt.legend()
10 plt.show()
```



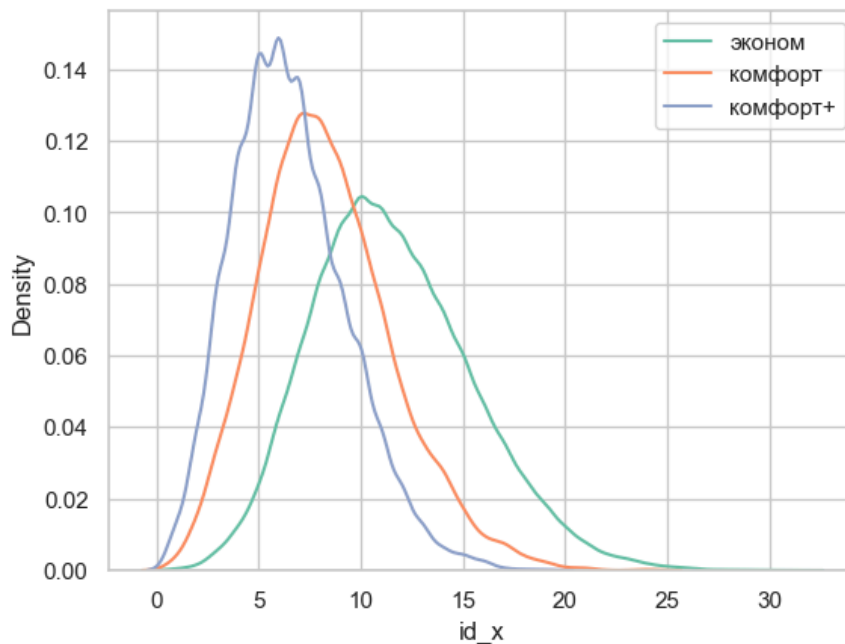
Посмотрим на распределение кол-ва заказов

Ввод [855]:

```

1 sns.kdeplot(orders_group_econom['id_x'], label='эконом')
2 sns.kdeplot(orders_group_comfort['id_x'], label='комфорт')
3 sns.kdeplot(orders_group_comfort_plus['id_x'], label='комфорт+')
4 # plt.xlim((0, 4000))
5 plt.legend()
6 plt.show()

```

**Вывод по задаче:**

- Доход водителя:
 - **Что видно:**
 - Для тарифа **эконом** (зелёный) пик плотности дохода ниже и уже, максимум — около 5 000 ₽.
 - Для **комфорт** и **комфорт+** кривая шире, хвост длиннее, доходы в среднем выше.
 - **Инсайт:** Таксисты на **комфорт** и **комфорт+** потенциально зарабатывают больше, но разброс больше — кто-то может уехать далеко и дорого, кто-то — просто сидит без заказов. Эконом более стабилен, но доход ниже.
- Доход компании с одного водителя
 - **Что видно:**
 - Тот же порядок: эконом < комфорт < комфорт+.
 - Разброс плотности тоже шире у более дорогих тарифов.
 - **Инсайт:** Для компании выгоднее, если таксист работает на более дорогом тарифе — комиссия и наценка растёт пропорционально среднему чеку.
- Доход компании со всех водителей тарифа
 - **Что видно:**
 - Тут неожиданно видно, что **эконом** даёт самый большой вклад — пик далеко вправо.
 - Комфорт и комфорт+ дают меньше.
 - **Инсайт:** Почему так? Потому что таких таксистов больше! Эконом массовый — значит, больше поездок и суммарный вклад в выручку больше, даже если с одного водителя она ниже. Это подтверждает логику агрегаторов: массовые дешёвые поездки — основа оборота.
- Заказы в день**
 - Эконом — больше заказов в день.
 - Это логично — поездки короткие, заказов много.
 - Комфорт+ — реже, но дороже.

Вывод

- Если ты — водитель, тебе выгоднее работать на более высоком тарифе, **если есть заказы и машина подходит**.
- Если ты — компания (агрегатор), тебе выгоднее массово продавать **эконом**, а премиум держать для лояльных клиентов и имиджа.
- Однако, большинство клиентов заказывают эконом

3. Итоги семинара

С чем выходим с занятия:

- вспомнили принципы работы с данными: ознакомиться с описанием данных, проверить данные на правдоподобность, проверить, есть ли дубли в данных, при соединении данных убедиться в правильности ключей, считать метрики так, как это принято у вас в компании
- посмотрели на принципы на примере
- попрактиковались в очищении данных
- сделали небольшое исследование и выводы из него

4. Полезные материалы:

- [Вредные советы при построении Аналитики \(https://habr.com/ru/companies/otus/articles/650471/\)](https://habr.com/ru/companies/otus/articles/650471/)
- [Про витрины данных \(https://www.decosystems.ru/data-mart/\)](https://www.decosystems.ru/data-mart/)